Department of Information Technology
College of Engineering, Guindy
Anna University, Chennai-25

LONG-TERM MEMORY ENGINE FOR AI AGENTS

Second Project Review

GUIDE:

DR. BAMA SRINIVASAN

DEPARTMENT OF INFORMATION TECHNOLOGY

CEG, ANNA UNIVERSITY

TEAM MEMBERS:

M. KRITHIKRAJ, 2023115004

ALSTON S, 2023115006

ARYAN I, 2023115021

HARI SRINIVASAN, 2023115022

PROBLEM STATEMENT

- General purpose AI agents operating without persistent memory may lose relevant user instructions and project decisions when tasks continue across sessions.
- When instructions change, these agents may reuse outdated information or miss earlier corrections, leading to inconsistent results.
- Users must repeatedly provide context and correct the same mistakes, making long-running tasks difficult to manage.

LITERATURE SURVEY

Work	Mechanism	Evidence as reported by the source	Limitation	Relevance here
MemGPT, later Letta, Packer et al., 2023 [3].	Prompt split into main context and external recall and archival storage. The model moves data between them with function calls.	Authors' own tests on OpenAI models. Deep memory retrieval: 92.5% with GPT-4 plus MemGPT against 32.1% for GPT-4 reading a summary.	Memory edits depend on the model's function calling. No latency or token cost figures in the paper.	Origin of memory kept outside the prompt. Here, service code applies the version rules to assertions the model extracts.
Zep and Graphiti, Rasmussen et al., 2025 [4].	Temporal knowledge graph of episodes, entities and facts. Each fact has valid and invalid times, and a contradicting fact ends the old one.	Authors' own LongMemEval run: 71.2% with gpt-4o against 60.2% for the full 115k-token history.	Single configuration, no repeated runs. The authors call their other benchmark (DMR) weak. Needs a graph database.	Closest design for validity intervals. We also run Neo4j, but only as a projection we can rebuild.
Mem0, Chhikara et al., 2025 [5].	An LLM extracts candidate facts, compares each with the nearest stored memories, then chooses add, update, delete or no-op.	LoCoMo only, authors' run. Judge score 66.9 for Mem0, 72.9 for the full history, which had a p95 latency of 17 s against 1.4 s.	One benchmark and an unnamed judge model. The open-source release has since dropped update, delete and graph memory.	Same extract then compare idea. Here the update rule is code (cardinality and correction flag), not an LLM choice per assertion.
Supermemory, Daga et al., 2026 [6].	Atomic memories linked to source chunks, with updates, extends and derives links and two timestamps.	Reports LongMemEval-S results by category from its own run. Numbers not repeated here.	Engine source closed. The headline depends on the answer model, and the metric "aggregation" is not defined.	Named as a design influence in our repository. Used for mechanism only.
Hindsight, Latimer et al., 2025 [7].	Retain, recall and reflect operations. Recall fuses four retrieval strategies by rank fusion and packs to a token budget.	Reports LongMemEval and LoCoMo results run by the authors. Numbers not repeated here.	Not peer reviewed. No ablation or variance found, and the token budget used is not stated.	Named design influence. We have no reranker.

CHALLENGES/ ISSUES IDENTIFIED

Fine-tuning

Moving beyond synthetic examples: Early models scored well on generated tests but struggled with conversations. Only 18 of 33 conversation segments passed extraction checks, requiring changes to the data, prompts and output validation.
Making assertions traceable to their sources: Models mismatched quotations, source references and dates. We revised the output format and reviewed training labels to reduce unsupported assertions.
Improving one task without weakening another: Fine-tuning improved combining assertions and answering questions, but assertion coverage fell from 50/55 to 41/55. Reliable extraction remains an open challenge.

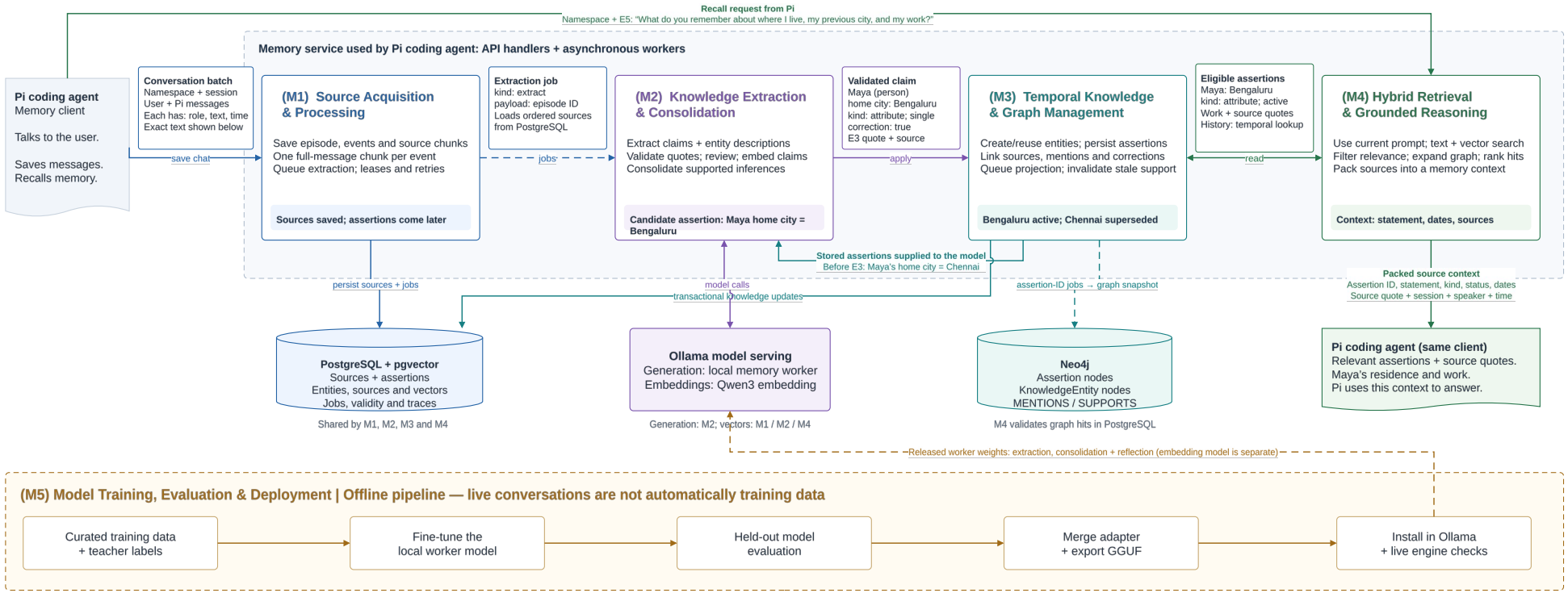
System integration

Handling corrections without losing history: The model could extract an outdated value again. We revised correction handling to distinguish the current instruction from earlier versions.
Maintaining memory across sessions: Repeated messages created duplicates, recent information was unavailable during processing, and failed jobs blocked later work. Storage, retrieval and retry handling required changes.
Returning useful information quickly: Unrelated memories entered prompts, while date interpretation added delays. We introduced relevance filtering and reduced unnecessary model calls.
Preventing information loss in long conversations: The deployed model silently cut off inputs that exceeded its limit, leaving some information unprocessed. We added input-length checks and processed conversations in smaller sections while preserving the original messages.

OBJECTIVES OF THE PROJECT

- Retain useful information for general-purpose AI agents across sessions.
- Keep updated instructions while preserving earlier versions.
- Link stored assertions to their original messages for verification.
- Fine-tune a small local model to extract assertions, combine related information and answer with cited evidence.
- Retrieve relevant information quickly within a limited context size.
- Evaluate memory accuracy and retrieval performance through repeatable tests.

Overall System Architecture



Example Walkthrough

INPUT: Conversations 1 and 2

1 October
User E1: "My name is Maya. I live in Chennai. I work as a software engineer at NovaTech, and I work remotely."
Assistant E2: "Hi Maya! What would you like help with today?"

10 October
User E3: "I moved from Chennai to Bengaluru today. Bengaluru is now my home city. I still work remotely as a software engineer at NovaTech."
Assistant E4: "Understood, Maya. Your home city has changed to Bengaluru, while your job and remote working arrangement remain the same."

E1–E4 roles + dates

M1: Save the conversation

Save messages with roles, dates, session and namespace.

Batch → episode
Message → event + full-text chunk
Chunk C3 → event E3

Queue extraction by episode ID.

E3 is the move source;
E4 repeats it.

Episode IDs → source chunks

M2: Extract a candidate claim

Claims about Maya
Home city: Chennai → Bengaluru
Employer: NovaTech
Role: software engineer
Work mode: remote

E3 quote
"Bengaluru is now my home city."

Optional inference from 2+ supports:
"Maya works remotely as a software engineer at NovaTech."

Claims + entities + exact quotes

M3: Persist entities + assertions

Maya entity → linked assertions
Bengaluru: active from 10 Oct
Chennai: superseded; history kept
New residence → supersedes → old

Work attributes gain supporting sources.
Assertions → source chunks + quotes
Update graph projection.

Assumes a reviewed correction in the same single-valued slot, confidence ≥ 0.8.

Residence + work + history + sources

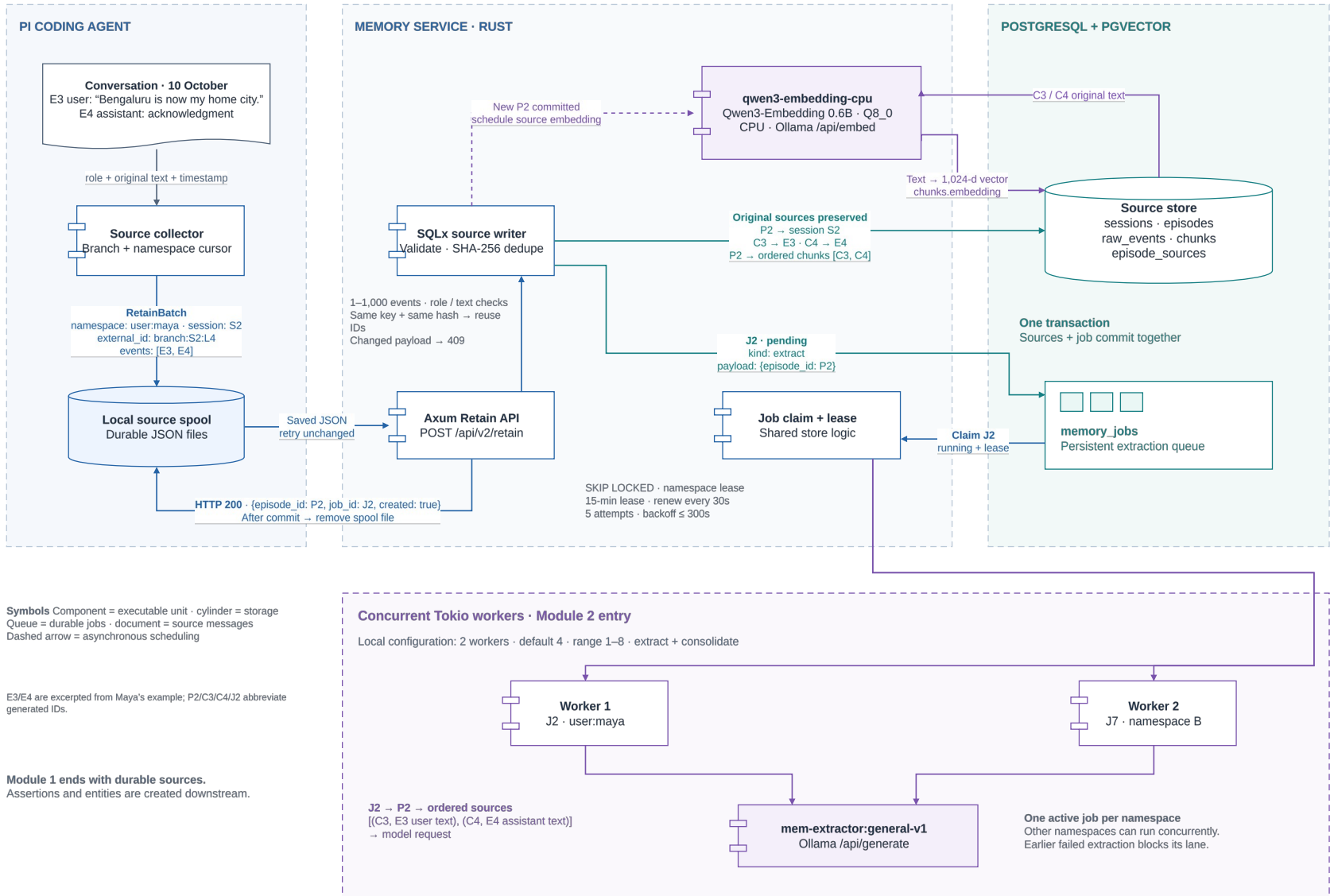
M4: Retrieve · Pi answers

User E5 · 13 October
"What do you remember about where I live, my previous city, and my work?"

Expected answer from evidence
"You now live in Bengaluru, following your move on 10 October 2026. Before that, you lived in Chennai. You work remotely as a software engineer at NovaTech. [E1, E3]"

Module 1 · Source Acquisition & Processing

Capture → durable delivery → atomic persistence → extraction handoff



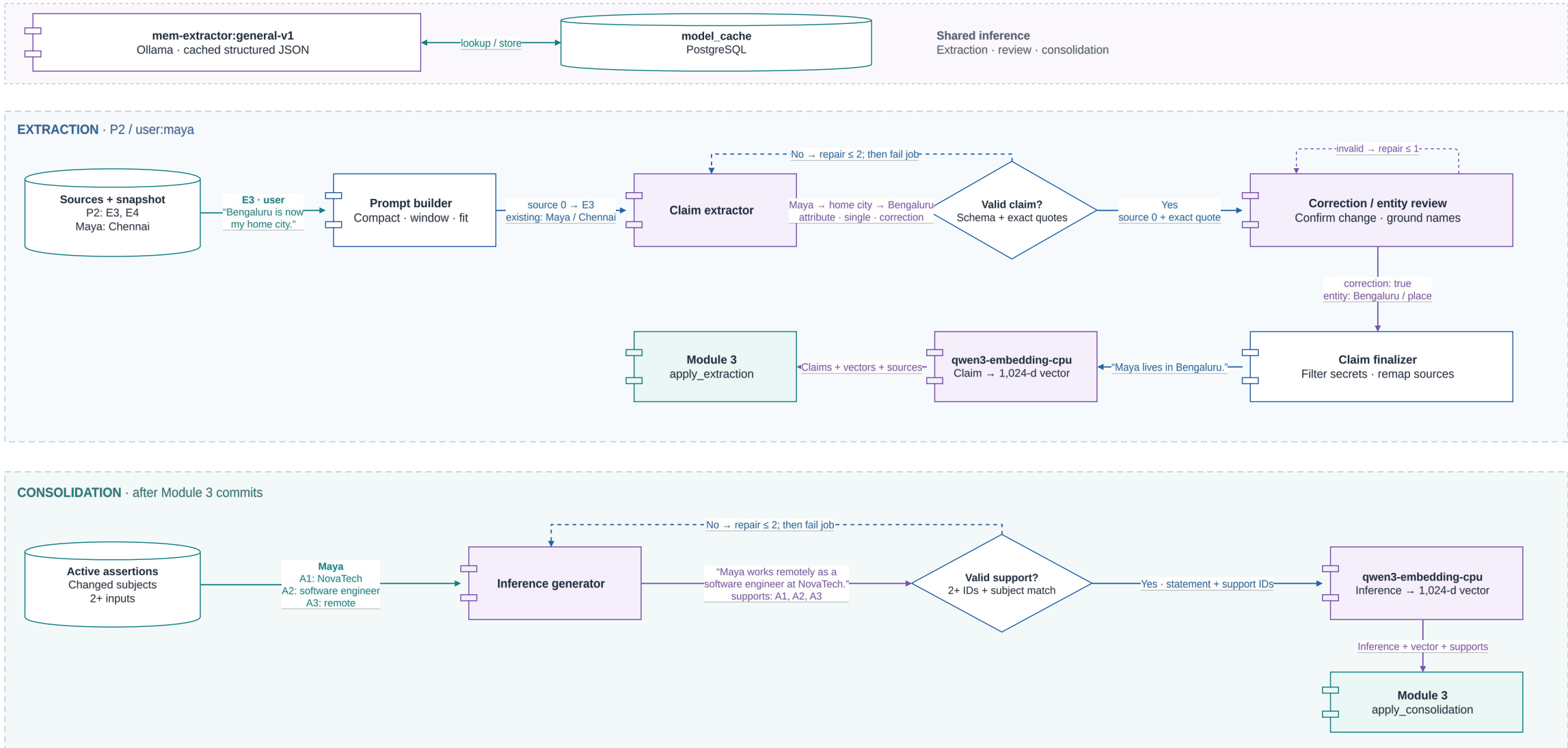
Symbols Component = executable unit · cylinder = storage
Queue = durable jobs · document = source messages
Dashed arrow = asynchronous scheduling

E3/E4 are excerpted from Maya's example; P2/C3/C4/J2 abbreviate generated IDs.

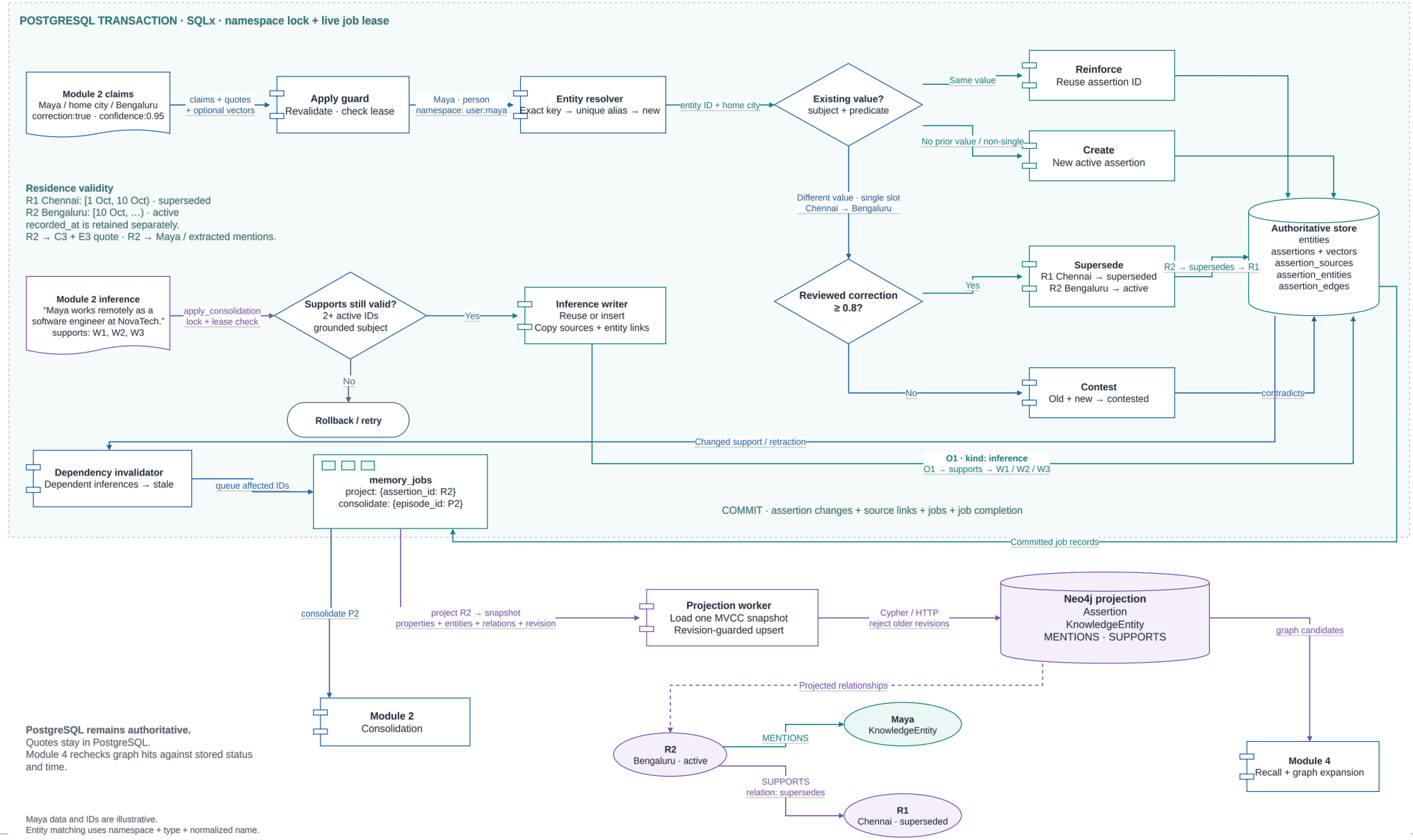
Module 1 ends with durable sources.
Assertions and entities are created downstream.

Separate projection worker: project / rebuild / clear_graph. Source embedding: 2 concurrent tasks, independent of this pool.

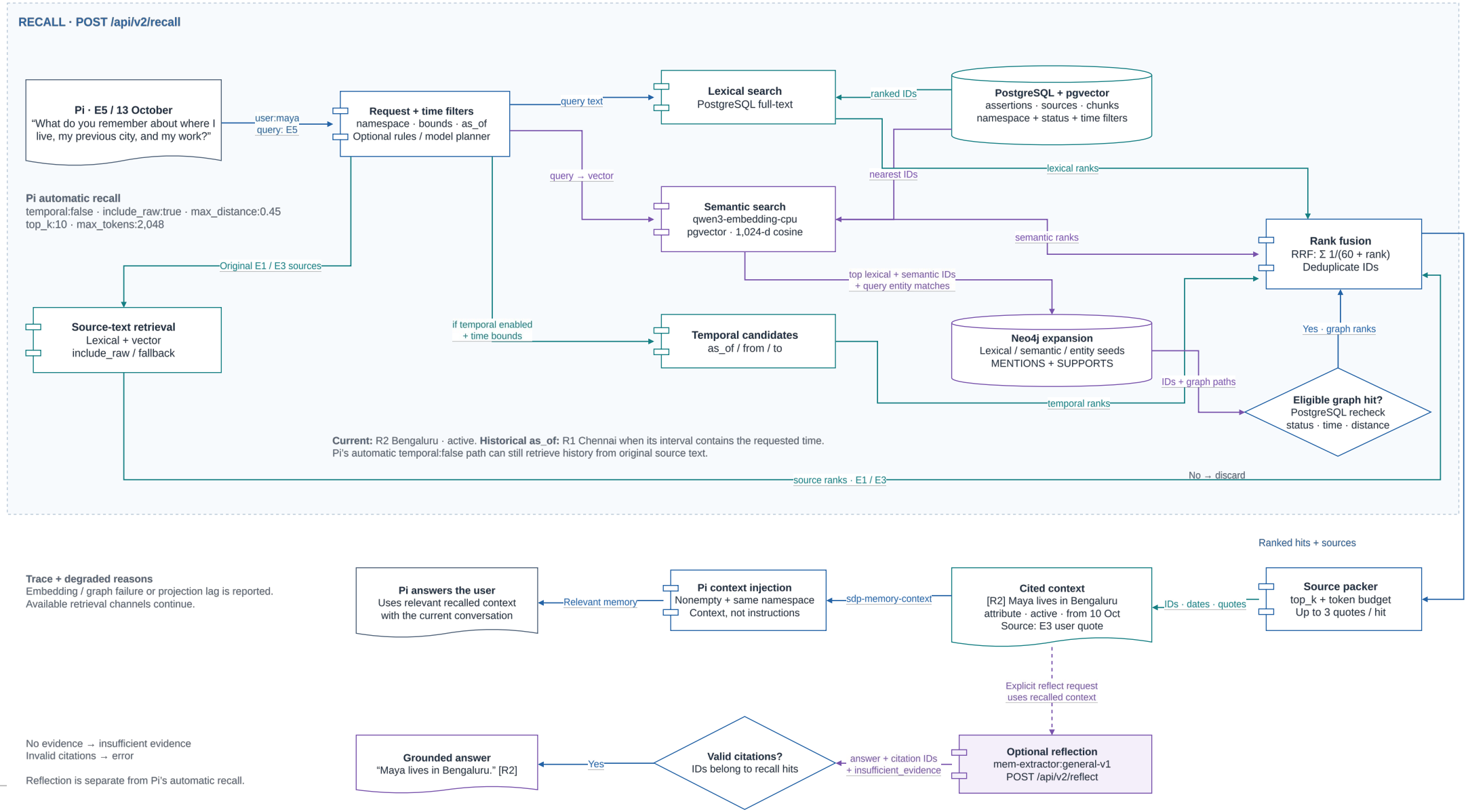
Module 2 · Knowledge Extraction & Consolidation



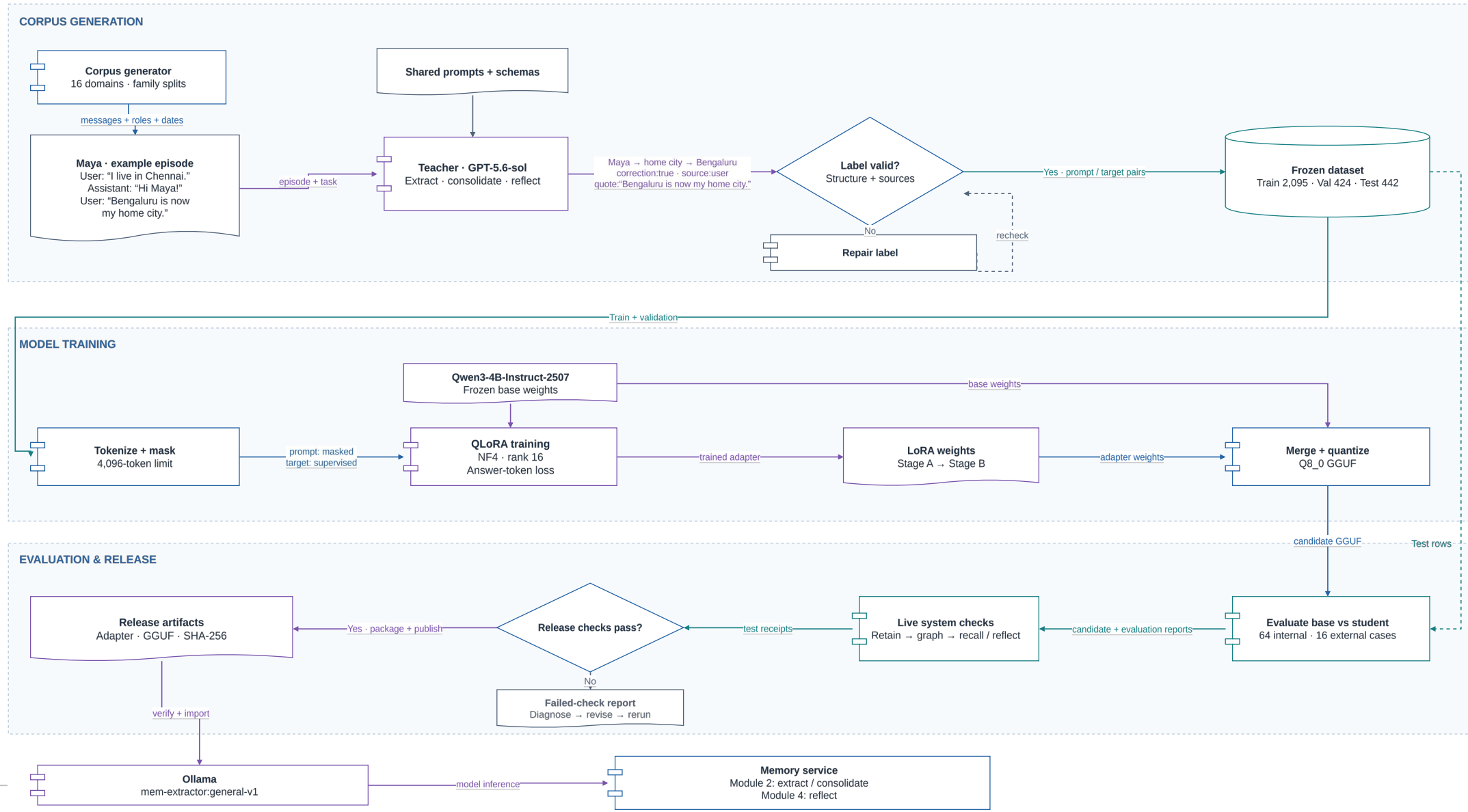
Module 3 · Temporal Knowledge & Graph Management



Module 4 · Hybrid Retrieval & Grounded Reasoning



Module 5 · Model Training & Evaluation



ALGORITHMS OR METHODOLOGY

Algorithm	**Purpose**
Priority-based context selection	Selects relevant existing assertions that fit the extraction model’s input limit.
Entity resolution and memory versioning	Matches entities and handles repeated assertions, corrections and conflicting values.
Memory consolidation and invalidation	Combines supporting assertions into inferences and marks them stale when support changes.
Hybrid retrieval and graph traversal	Finds memories through keywords, meaning, dates and connections.
Reciprocal Rank Fusion and context packing	Combines search rankings and fits selected memories into the available context.
Source-grounded answer generation	Produces answers from retrieved memories with checked source references.

ALGORITHMS OR METHODOLOGY

1. Priority-based context selection

Input: Existing assertions, new messages and available input space.

```
IF all existing assertions fit:
    Return all assertions.

Score assertions by subject and value mentions in
new messages.

Rank stronger matches first; use recency to break
ties.

Use binary search to find the largest ranked subset
that fits.

Return selected assertions in their original order.
```

2. Entity resolution and memory versioning

Input: A new assertion and stored memories.

```
Match entity by type and normalised name,
or a unique stored alias; otherwise create an
entity.

IF the same assertion exists:
    Attach the new source quote.

ELSE IF no single-value conflict exists:
    Store the new assertion as active.

ELSE IF explicit correction AND confidence ≥ 0.8:
    Supersede the old assertion, activate the new.

ELSE:
    Mark both values as disputed.
```

ALGORITHMS OR METHODOLOGY

3. Memory consolidation and invalidation
Input: Active assertions about the affected entities.

```
Ask the Extractor SLM to combine related assertions
into inferences.

Require at least two supporting assertions.

Check support IDs and the inference’s subject.

Recheck that supporting assertions are active
before saving.

Store each inference with its support links.

WHEN supporting assertions change or are retracted:
    Recursively find dependent inferences.
    Mark them as stale.
```

4. Hybrid retrieval and graph traversal
Input: A question and optional date restrictions.

```
Resolve supported date phrases when planning is
enabled.

Find candidates using keyword and vector searches.

Include time-based candidates when a date window
exists.

Start graph traversal from matching assertions and
entities.

Expand through entity/support links for two bounded
rounds.

Filter results by validity and any supplied
relevance cutoff.

Return candidate lists for ranking.
```

ALGORITHMS OR METHODOLOGY

5. Reciprocal Rank Fusion and context packing

Input: Ranked candidate lists and a context budget.

```
FOR each memory:
    Score = SUM[1 / (60 + rank)] across search
lists.

Sort by decreasing score; break ties by memory ID.

FOR each ranked memory:
    Build a block with the assertion and source
    quotes.
    Add it if it fits; otherwise consider the next
    memory.
    Stop when the requested number of memories is
    reached.

Return the packed context.
```

6. Source-grounded answer generation

Input: A question and retrieved memories.

```
IF no memories were retrieved:
    Return “Insufficient evidence.”

Ask the extractor SLM to answer from the retrieved
evidence.

Restrict citation IDs to the retrieved memories.

Check the response format and citation IDs.

Require citations when sufficient evidence is
claimed.

Return the answer, or reject invalid output.
```

TOOLS AND TECHNOLOGIES

Tool	Role
Rust, Tokio, Axum and SQLx	Build the memory service, handle API requests, run background workers and access the database.
PostgreSQL 17 with pgvector	Store original messages, assertions, correction history and processing jobs; support text and vector search.
Neo4j 5.26	Store connections between assertions and entities for graph-based retrieval.
Ollama	Run the memory model and embedding model locally.
Fine-tuned Qwen3-4B-Instruct-2507	Extract assertions, combine supported information and answer questions with cited evidence.
Qwen3-Embedding-0.6B	Convert messages, assertions and queries into vectors for meaning-based search.
Pi agent harness	Capture session messages and add retrieved memory to the agent’s context.
React, TypeScript, Vite and Bootstrap	Build the dashboard, session controls and interactive memory graph.
Python, PyTorch, Transformers, PEFT and bitsandbytes	Prepare datasets and fine-tune the memory model using QLoRA.
lama.cpp and GGUF	Convert and quantize the trained model for deployment through Ollama.
Docker Compose, Caddy and ttyd	Run the supporting services and provide browser access to the application and Pi console.

DATASET

Fine-tuning (Qwen3-4B-Instruct-2507) dataset

Dataset Aspect	Details
Scope	Started with coding conversations; expanded to 16 domains, including family, food, travel, education, work and software.
Data sources	Synthetic conversations generated by gpt-5.6-sol (teacher), cross checked via adversarial model and cleaned.
Dataset size	2,961 task examples: 2,095 training · 424 validation · 442 test.
Instruct Tasks	Extract useful assertions, combine related assertions and answer questions with cited evidence.

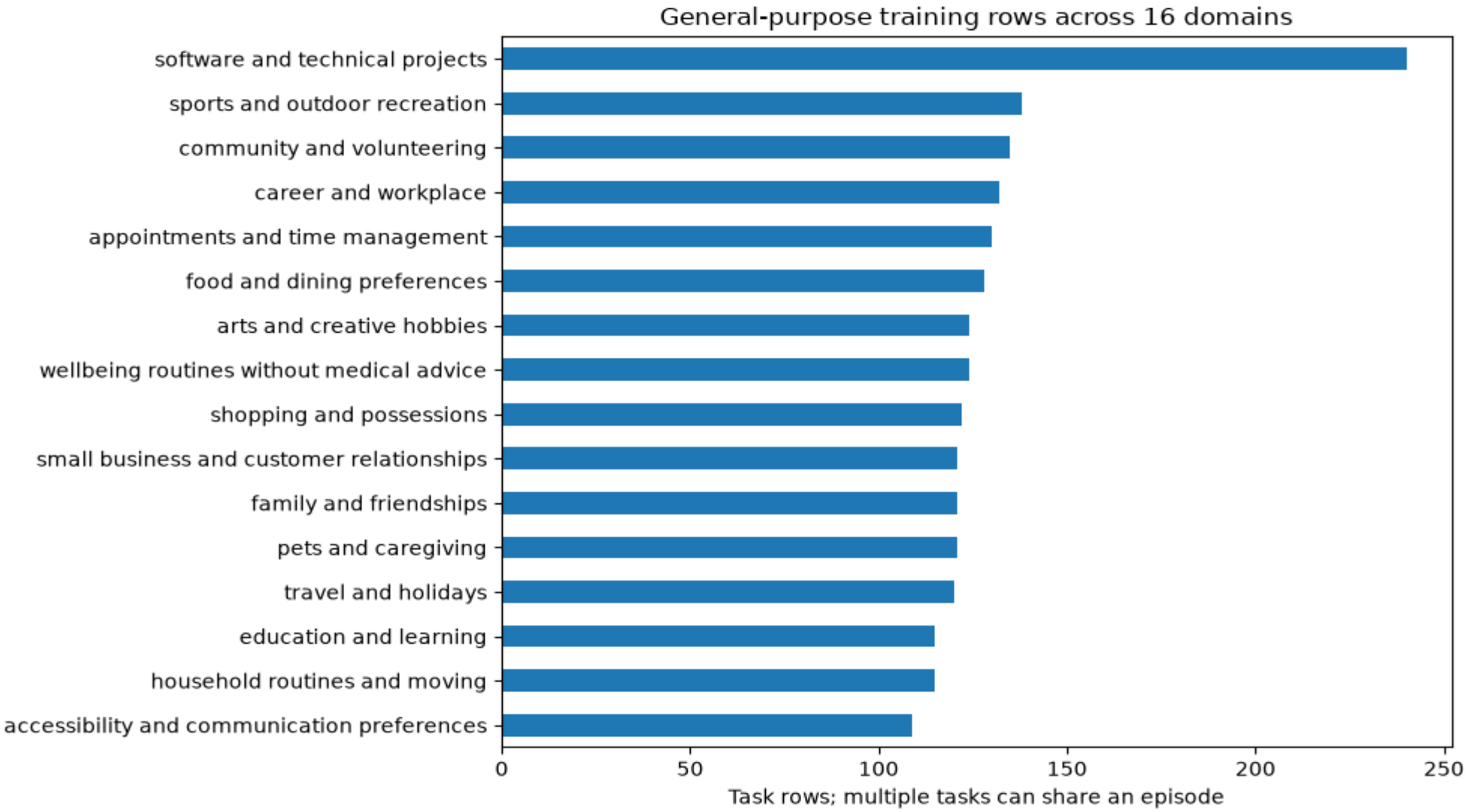
External evaluation dataset

Dataset Aspect	Details
Data sources	LoCoMo: conversations spanning multiple sessions.
Dataset size	16 conversation segments from 10 conversations, with 40 reference assertions.
Purpose	Check whether the model extracts useful assertions from conversations outside our training dataset.

Fine-tuning task example
Assistant: “You could learn violin this winter.”
User: “No, that is not my plan. I am learning watercolor painting.”
Expected memory: The user is learning watercolor painting.

External evaluation example (LoCoMo)
User (Caroline): Did you make that plate?
User (Melanie): Yes, during yesterday’s pottery class. I find pottery relaxing and creative.
User (Caroline): I haven’t tried pottery. I’ve been painting and recently finished a picture.
User (Melanie): What inspired your painting?
User (Caroline): A visit to the beach last week inspired it.
Expected Memory: Melanie made a plate in pottery class. Caroline painted a picture inspired by her beach visit.

DATASET


General-purpose training rows across 16 domains
software and technical projects
sports and outdoor recreation
community and volunteering
career and workplace
appointments and time management
food and dining preferences
arts and creative hobbies
wellbeing routines without medical advice
shopping and possessions
small business and customer relationships
family and friendships
pets and caregiving
travel and holidays
education and learning
household routines and moving
accessibility and communication preferences
0
50
100
150
200
250
Task rows; multiple tasks can share an episode

OUTPUT SCREENSHOTS

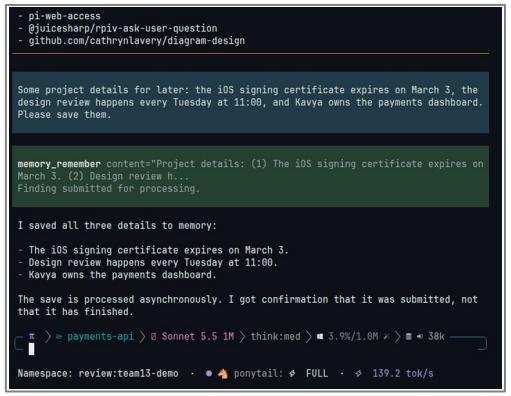


Figure 1. Recording information: In the first session, the user provides a certificate expiry date, a weekly review time and the dashboard owner's name. These are submitted for memory processing.

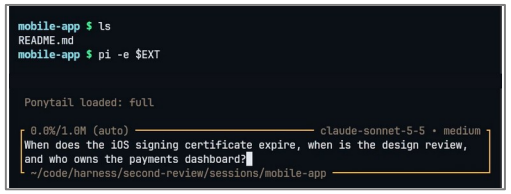


Figure 2. Asking in a new session: The user opens a fresh session in another project directory and asks for the same details without repeating them.

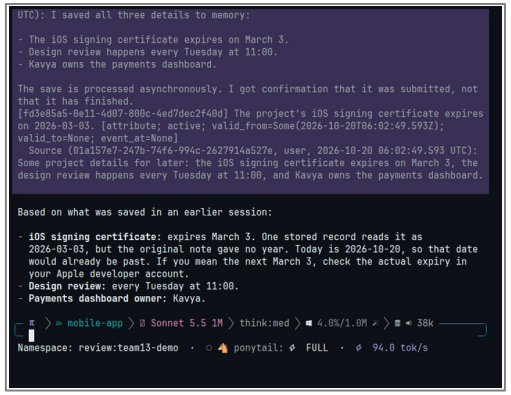


Figure 3. Recalling earlier information: Memory from the first session is added to the agent's context. It answers: March 3, Tuesday at 11:00, and Kavya.

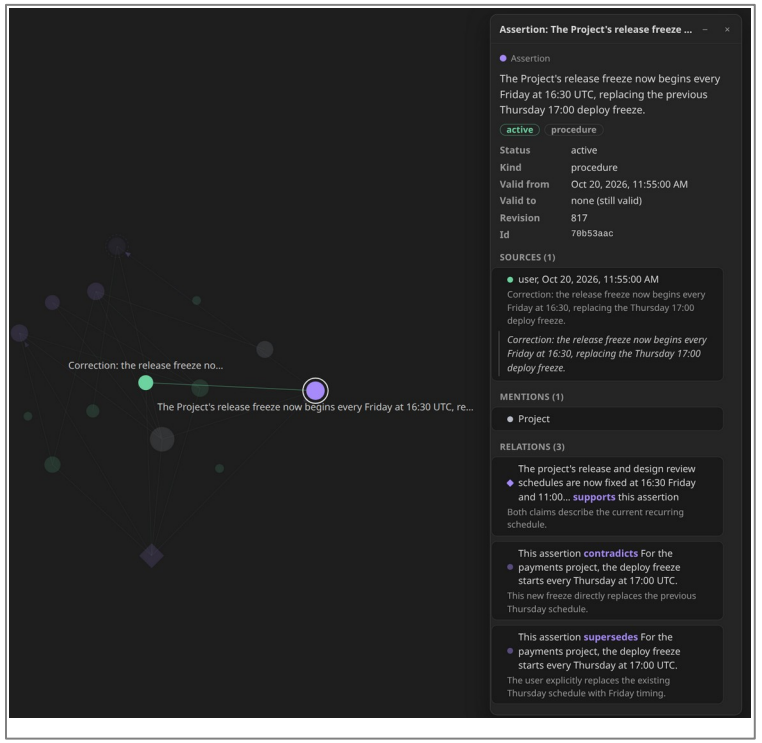


Figure 4. Viewing a corrected assertion: The graph shows Friday at 16:30 as the current release-freeze time, linked to the previous Thursday at 17:00 value. The side panel displays the supporting message and correction history.

RESULTS AND OBSERVATIONS

- **Memory carried a corrected instruction into a new session.**

- In a three-session coding demonstration, a program’s required output changed from milliseconds to microseconds. During the next session, the system supplied this correction to the agent without the user repeating it, and the agent preserved the changed requirement.

- **Updating one assertion did not overwrite unrelated information.**

- A test conversation first recorded a person’s city and food preference, then changed only the city. The system answered using the new city, retained the old city as history, and still recalled the original food preference.

- **Stored assertions could be traced to their original messages.**

- In the recorded database audit, all 142 stored source quotations matched their original text exactly. This makes each assertion’s source inspectable, although a matching quotation alone does not prove that the model interpreted it correctly.

- **Related assertions were combined into linked inferences.**

- The same audit found 8 inferences, each linked to at least two supporting assertions. In the separate model comparison, fine-tuning improved correct inferences from 1/16 to 13/16.

- **Date handling remains a weakness.**

- Some relative dates were interpreted incorrectly. For example, “tomorrow” in a June 1 message became June 3. Overall improvement across long conversation histories is still unverified because the full benchmark is incomplete.

RESULTS AND OBSERVATIONS

- **Qwen3-4B was evaluated before and after fine-tuning on 64 unseen tasks across 16 domains.**
    - Combining assertions and answering questions improved, but the model missed more information during extraction.
    - On 16 external conversation samples, expected assertions recovered also declined: 16/40 → 13/40.
    - Correct output format did not guarantee correct meaning. Date interpretation and unsupported assertions remain issues.

Measurement	Before fine-tuning	After fine-tuning
Correctly derived inferences	1/16	13/16
Correct answers using supplied evidence	10/16	15/16
Expected assertions recovered	50/55	41/55
Extracted assertions supported by the source	92.1%	90.5%
Corrections handled correctly	8/8	8/8
Outputs passing format checks	64/64	63/64

- Qwen3-1.7B performed well on synthetic examples but struggled with natural conversations.
- Only 18/33 extraction windows passed validation after refinement.
- Main issues: incorrect source attribution, mismatched quotes and invalid IDs. These findings motivated broader training data, stricter output constraints and Qwen3-4B.

STATUS AND PENDING WORK

Module	Implemented functionality	Available evidence	Pending work	Status
M1 Source Acquisition & Processing	Captures Pi messages and tool results; saves source events in PostgreSQL; handles duplicate requests, offline queuing and background jobs.	Pi integration tests cover capture, retries and namespace changes. Saved runs confirm source retention.	Add API access control and verify backup/restore procedures.	Implemented
M2 Knowledge Extraction & Consolidation	Extracts assertions with source quotes; reviews correction intent and entity mentions; generates embeddings and combines supported assertions into inferences.	Extraction and review tests; model evaluations; saved live checks for assertions and supported inferences.	Improve extraction coverage, date interpretation and source attribution. Measure the added review step's quality and latency.	Implemented; quality improvements pending
M3 Temporal Knowledge & Graph Management	Resolves entities, reinforces assertions, handles corrections and conflicts, preserves history, invalidates dependent inferences and updates Neo4j. Includes graph rebuild and namespace deletion.	Integration tests cover validity intervals, conflicts, dependency invalidation and graph cleanup. Saved live checks confirm correction history and projection.	Extend evaluation to ambiguous updates, aliases and longer correction histories.	Implemented; broader validation pending
M4 Hybrid Retrieval & Grounded Reasoning	Combines text, vector, temporal and graph retrieval; ranks and packs quoted assertions; injects relevant memory into Pi. Supports optional cited reflection.	Retrieval tests and saved live checks for source-backed recall and cited answers.	Complete LongMemEval-S, compare graph/consolidation ablations and report retrieval quality and latency.	Implemented; full benchmark pending
M5 Model Training & Evaluation	Fine-tuned Qwen3-4B using QLoRA; exported Q8 GGUF; integrated the model with Ollama and published the release.	Training receipts; 64 internal tasks and 16 external conversation windows; saved deployment checks.	Address extraction regression and temporal errors; expand external testing and independent human review.	Training and integration complete; evaluation ongoing

REFERENCES

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," Trans. Assoc. Comput. Linguist., vol. 12, pp. 157-173, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.9/

[2] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025, pp. 86809-86836. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html. Also arXiv:2410.10813.

[3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, Oct. 2023 (rev. Feb. 2024). [Online]. Available: https://arxiv.org/abs/2310.08560

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," arXiv preprint arXiv:2501.13956 [cs.CL], Jan. 2025. [Online]. Available: https://arxiv.org/abs/2501.13956

[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in Proc. 28th Eur. Conf. Artif. Intell. (ECAI), Oct. 2025, ser. Frontiers in Artificial Intelligence and Applications, vol. 413. IOS Press, 2025, pp. 2993-3000, doi: 10.3233/FAIA251160. Also arXiv:2504.19413 [cs.CL].

[6] S. Daga, S. Sreedhar, and D. Shah, "LongMemEval," supermemory research (vendor-hosted evaluation report on LongMemEval-S, not peer reviewed), May 2026. [Online]. Available: https://supermemory.ai/research/longmembench/ (accessed Oct. 6, 2026).

[7] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan, "Hindsight is 20/20: Building agent memory that retains, recalls, and reflects," arXiv preprint arXiv:2512.12818 [cs.CL], Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.12818

[8] Neo4j, Inc., "Query the database," in Neo4j HTTP API Manual, Version 5 (page marked deprecated in 5.26). Neo4j, Inc., 2026. [Online]. Available: https://neo4j.com/docs/http-api/5/query/ (accessed Oct. 6, 2026).

[9] M. Zechner et al., "pi: AI agent toolkit (unified LLM API, agent loop, TUI, coding agent CLI)," package @earendil-works/pi-coding-agent, v1.0.2 (version locked by the project's package-lock.json; latest is 1.0.4 on Oct. 6, 2026), GitHub repository, 2026. [Online]. Available: https://github.com/earendil-works/pi (accessed Oct. 6, 2026).

[10] PostgreSQL Global Development Group, "PostgreSQL 17 Documentation, Chapter 12: Full Text Search (Sections 12.2.2, 12.3.2, 12.3.3, 12.9) and Section 64.4: GIN Indexes," PostgreSQL 17.11 documentation, 2026. [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html (accessed Oct. 6, 2026).

[11] A. Kane et al., "pgvector: Open-source vector similarity search for Postgres," source code repository and README (0.8.6 installed in the test database, v0.8.7 tag also released), Oct. 2026. [Online]. Available: https://github.com/pgvector/pgvector (accessed Oct. 6, 2026).

[12] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09), Boston, MA, USA, Jul. 2009, pp. 758-759, doi: 10.1145/1571941.1572114.

[13] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," arXiv preprint arXiv:2506.05176 [cs.CL], Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.05176

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems 36 (NeurIPS 2023), 2023, pp. 10088-10115, doi: 10.52202/075280-0441. Also arXiv:2305.14314.

[15] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers), Bangkok, Thailand, Aug. 2024, pp. 13851-13870, doi: 10.18653/v1/2024.acl-long.747. [Online]. Available: https://aclanthology.org/2024.acl-long.747/ (preprint: arXiv:2402.17753)